Software Design Description (SDD)

Pauline Kang, Aaron Tsai, Nicholas Kicinski, Tani Dharmendran

- Frontispiece - before the sections start
    - Date of issue and status: November 2025, Draft Version 1.0
    - Issuing organization: Fundraising Website Development Team
    - Authorship: Pauline Kang, Aaron Tsai, Nicholas Kicinski, Tani Dharmendran
    - Change history

Version	Date	Description	Author(s)
1.0	Nov 2025	Initial Draft	Pauline, Aaron, Nicholas,Tani

- section 1: SDD Identification (see clause 4.2)
    - 1.1: Summary/overview : This document explains how the Fundraising Website is designed and how it will work. It covers the system structure, data flow, and main components so the development team has a clear plan. The goal is to make sure the website works well for students, donors, and administrators and makes organizing fundraisers simple.
    - 1.2: Scope: The SDD focuses on the technical design of the Fundraising Website. It describes how the website handles fundraisers, donation tracking, and user input. This system is intended for school and local community use and aims to give users a simple, easy way to manage campaigns and see fundraising progress.
    - 1.3: Context: The website is a standalone application that runs on a browser using local hosting. It connects a user-friendly front-end with a database that stores fundraiser and donation information. Users can create campaigns, track progress, and submit donations. It is designed to be simple, reliable, and functional for its intended audience.
    - 1.4: Design Language - We are using UML to describe the system design. This includes diagrams like class diagrams, sequence diagrams, and use case diagrams to show how different parts of the system work together. The front end will be built with HTML,

CSS, and JavaScript and follow a Model-View-Controller (MVC) design.

- 1.5: Glossary - include UML (could also include GUI, SRS, HTML, etc.)

Term	Definition
UML	Unified Modelling language used for diagrams showing system structure and behavior
GUI	Graphical User Interface, what the user sees and interacts with
SRS	Software Requirements Specifications, Documents that explains what the software must do
HTML	For building web pages
CSS	Cascading style sheets, for styling web pages
JavaScript	Language used to add logic and interactivity to web pages

- 1.6: References : IEEE 1016:2017

- section 2: Design stakeholders and their concerns (see clause 4.3)

Stakeholder	Role	Concerns
Donors	End Users	Ease of use, data security, secure transactions
Administrators	Campaign managers	Data accuracy, analytics tools
Developers	Implementation team	Scalability, maintainability

School Orgs	Oversight and compliance	Legal compliance, financial transparency

- section 3: Design viewpoints (see clause 4.5)
    - 3.1: Introduction - list what viewpoints will be used

    We will be using the logical viewpoint and the physical/deployment viewpoint.

    - 3.2: Design viewpoint 1 - how does this viewpoint apply to your software?
        - 3.2.1: Design concerns - Component interactions and data flow between frontend, backend, and database.
        - 3.2.2: Design View - The system will follow a Model-View-Controller, where the model manages campaign, user, and transaction data while the view is a React-based interface that displays the stats. Finally, the controller handles routing and input validation.
        - 3.2.3: Design Elements (see clause 4.6) - React, CSS, Django REST Framework (if time allows), PostgresQL?, authentication tokens?
        - 3.2.4: Design Overlays (see clause 4.7) - as applicable - Encryption of payment data through tokenization
        - 3.2.5: Design Rational (see clause 4.8) - React and Django were chosen for scalability, community support, and compatibility with RESTful API (if time allows, although I doubt it)> PostgresQL should ensure consistency?
    - 3.3: Design viewpoint 2 - How does this viewpoint apply to your software?
        - 3.3.1: Design concerns - Hosting reliability, scalability, and maintenance.
        - 3.3.2: Design View - The system will be deployed locally. Maybe in the future through Docker containers for reproducibility.
        - Design Elements - Just local. GitHub for CI/CD for testing and deployment
        - Design Overlay - Current